2022.03.01.

4. Gyakorlat

Láncolt listák

- dinamikus adatszerkezet, a memóriában nem folytonosan vannak tárolva
- a rekordoktól a rákövetkezőkre pointerek vezetnek
- egy vagy kétirányú listák
- fejelemes vagy fejelem nélküli

Egyirányú lista

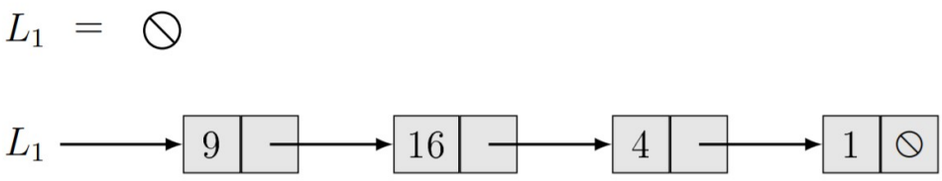
- *egyszerű egyirányú listák (S1L - Simple 1-way List)*
- *fejelemes egyirányú listák (H1L - Header node + 1-way list*)

Listaelem típusa

E1
$+key : \mathcal{T}$ $\ldots$ // satellite data may come here $+next$: **E1***
$+\mathbf{E1}()\ \{\ next := \oslash\ \}$

- `El*` olyan pointert jelöl, ami `El` típusú objektum címét tartalmazza (vagy null, vagy definiálatlan)
- a lista bejárásához pointereket használunk:
  - `p:El*` pointer
    - a mutatott objektum adattagjai:
      - `p -› key`
      - `p -› next`
- új listaelem létrehozása:
  - `p := new El`
- felesleges listaelem felszabadítása:
  - `delete p`

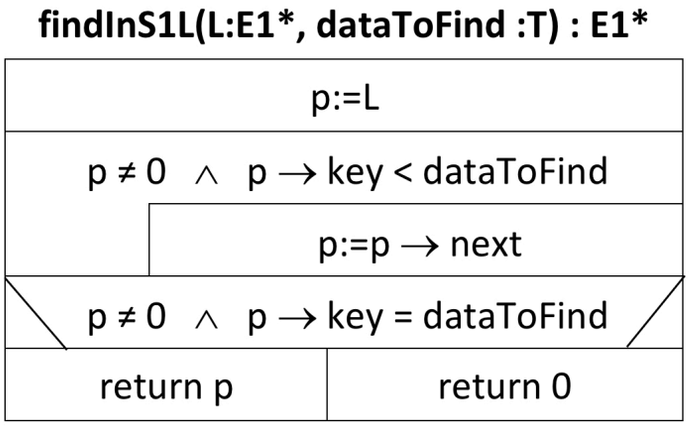
Egyszerű egyirányú láncolt lista - S1L



S1L műveletei

- keresés kulcs alapján
  - az elem benne van-e a listában
- beszúrás
  - egy elem a listához adása (ha még nincsen benne)
- törlés
  - egy elem kivétele a listából (ha benne van)

Keresés



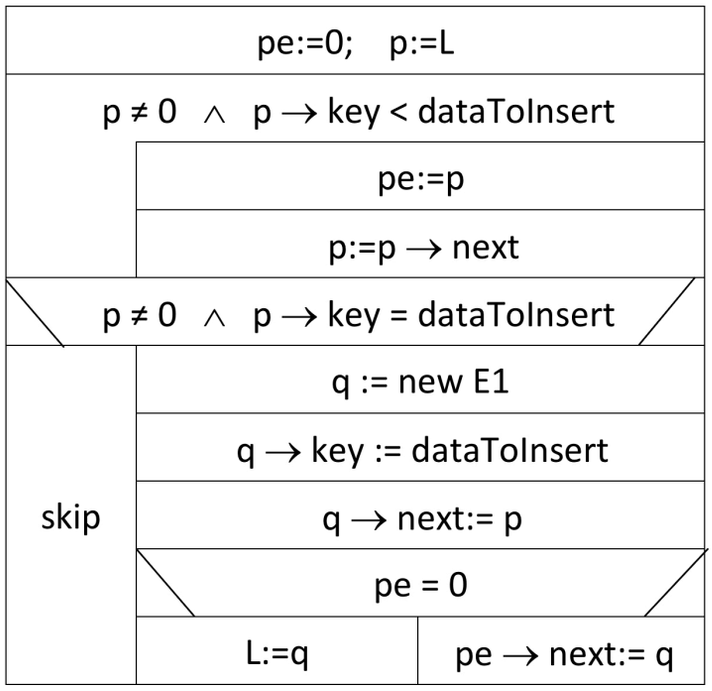
Beszúrás

Esetei:
- lista belsejébe szúrunk be
- lista utolsó eleme után szúrjuk be az új elemet
- lista elejére szúrunk be
- üres listába beszúrás

A műveletekhez két elem címe kell:

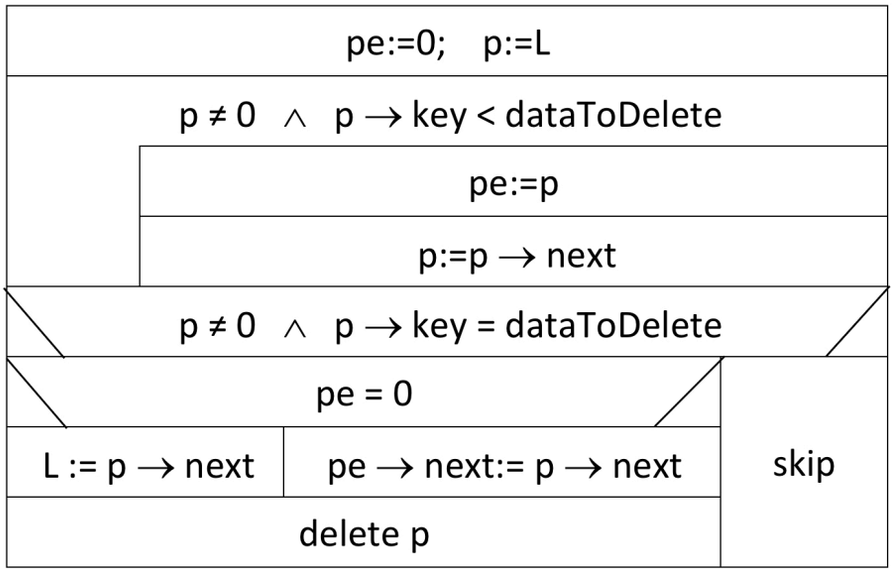
- amelyik elé fogunk befűzni
- az előtte levő elem címe

insertIntoS1L(&L: E1*, dataToInsert: T)

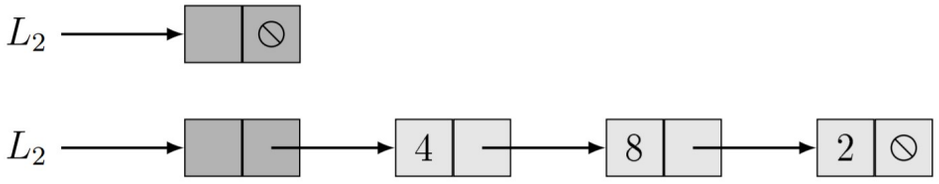


Törlés

deleteFromS1L(&L:E1*, dataToDelete: T)



Fejelemes egyirányú láncolt lista - H1L



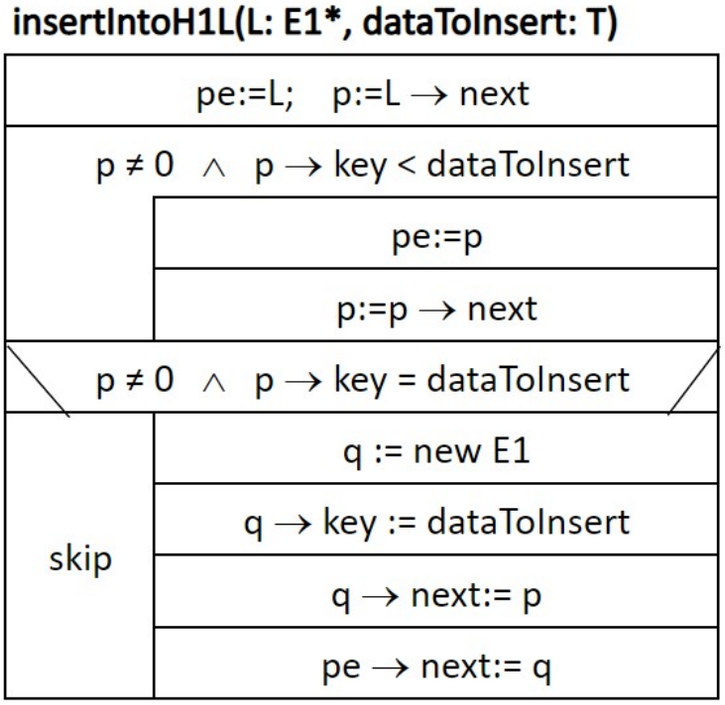
H1L műveletei

Keresés

- induló lépés:
    - p := L -› next (a fejelemben nincs kulcs)

Beszúrás

- indulásnál pe a fejelemre, p a lista első elemére mutat



Törlés

- indulás a beszúráshoz hasonlóan

Egyszerű lista megfordítása

Trükk: Mintha egy verembe raknánk a lista elemeit, az eredmény listának mindig az elejére fűzzük be az eredeti lista aktuális elemét.

megfordít(&L:E1*)

p:=L
L := null
p ≠ null

q := p
p := p -> next
q -> next := L
L := q

H1L szétfűzése két listába

Adott egy egész számokat tartalmazó H1L, a fejelemre L1 mutat. Fűzzük szét az elemeket két listába:

- L1-ben maradjanak a páros elemek,
- egy új L2 fejelemes listába fűzzük át a páratlan elemeket.

Az elemek eredeti sorrendjét tartsuk meg. Az L1 listát egyszer lehet bejárni, L2-be a befűzés konstans műveletigényű legyen.

Szétfűz(L1:E1*): E1*

<table>
<tr><td colspan="3">L2 := new E1 //L2 → next=0</td></tr>
<tr><td colspan="3">u:=L2</td></tr>
<tr><td colspan="3">pe:= L1; p:=L1 → next</td></tr>
<tr><td colspan="3">p ≠ 0</td></tr>
<tr><td></td><td colspan="2">p → key mod 2 = 0</td></tr>
<tr><td></td><td>pe:=p</td><td>pe → next:= p → next</td></tr>
<tr><td></td><td rowspan="4">p:= p → next</td><td>u → next:= p</td></tr>
<tr><td></td><td>p → next:= 0</td></tr>
<tr><td></td><td>u:=p</td></tr>
<tr><td></td><td>p:= pe → next</td></tr>
<tr><td colspan="3">return L2</td></tr>
</table>

Első lépésként egy új H1L létrehozását mutatjuk be. Kihasználjuk, hogy E1 konstruktora az elem next pointerét 0-ra állítja!

u mindig L2 utolsó elemére fog mutatni, ez kezdetben a fejelem.

pointerek L1 bejárásához

Ha páratlan kulcsú elemmel találkozunk, a lépések:

p kifűzése

p befűzése L2 végére

mivel p az L2 utolsó eleme, 0-ra állítjuk a next pointerét

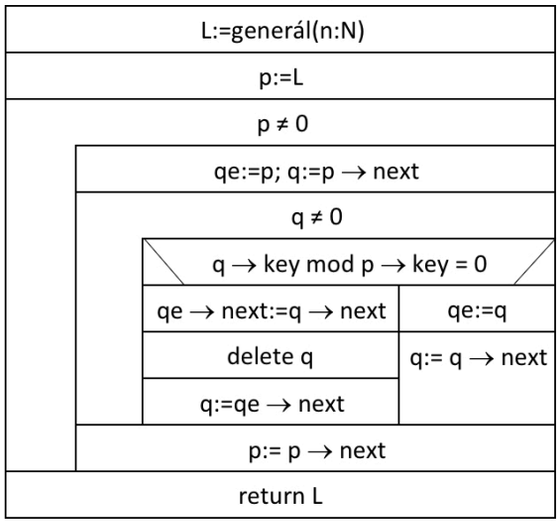
módosítjuk u-t

p-vel L1 következő elemére állunk

Prímszita egyszerű listán

Készítsünk egy listát, mely 2-től n-ig tartalmazza a prímeket, az Eratoszthenészi szita algoritmus ötletét felhasználva. Töltsünk fel egy egyszerű listát 2..n természetes számokkal.
Az első elem prím, azokat, melyek oszthatók ezzel az elemmel, töröljük a listából. A következő megmaradt szám megint prím. Többszöröseit ismét töröljük. A végén a prímek maradnak a listánkban.

Primszita(n:N) : E1*



Generálunk egy egyszerű listát 2..n természetes számokból.

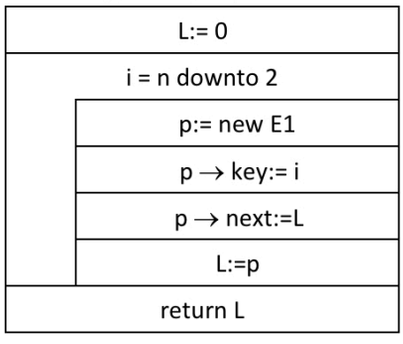
p mindig a következő, még a szitában lévő prímszámra mutat (elsőként a 2-re). Amíg ez nem 0 …

q-val a p utáni elemeket fogjuk bejárni, és kifűzzük p → key többszöröseit

Ha p→key osztója q→key-nek, a q című elemet töröljük a láncból. Ehhez szükségünk van q elem előzőjére, ez lesz a qe pointerben.

ha p című elem többszöröseit töröltük, a következő elem megint prím lesz.

generál(n:N) : E1*



Egyszerű lista feltöltése 2..n természetes számokkal.
Trükk: a ciklust csökkenőleg futtatjuk, így mindig a lista elejére kell befűzni.